Name: Nguyễn Dương Phúc
Student ID: 24521386
Class: CS4323.Q12.CTTT

OS
LAB 4

CHECKLIST

1. Homework

	1	2
Explanation	☒	☒
Your code	☒	☒
Results and discussions	☒	☒

Mark: 10/10 (self-scored)

**Notice: Export PDF file, name format:*
<MSSV>_LAB4.pdf

2. Lab 4

1. Write a program time.cpp that measures the execution time of a shell command. The program will be executed with the syntax ./time <command> where <command> is the shell command you want to measure the execution time for.

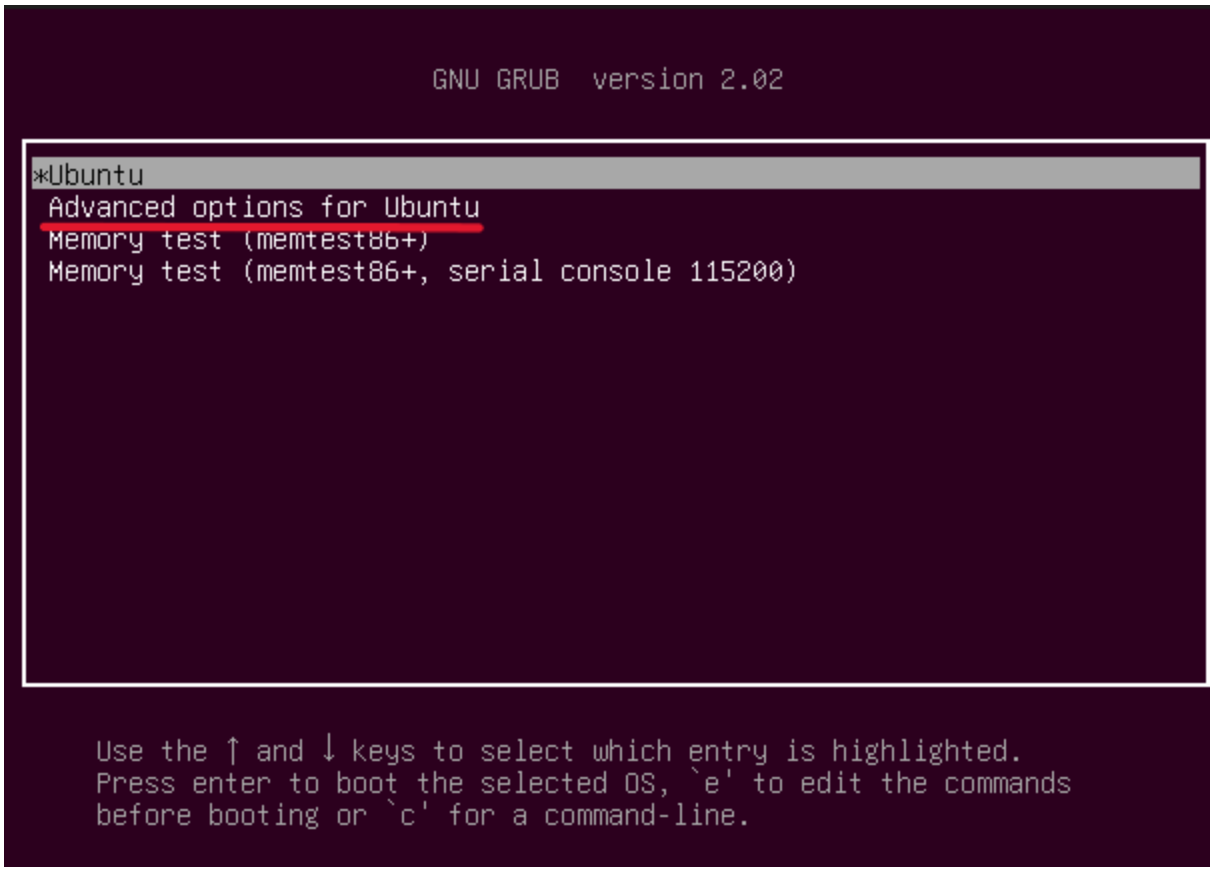
Example:

```
$ ./time ls
time.c
time
Execution time: 0.25422
```

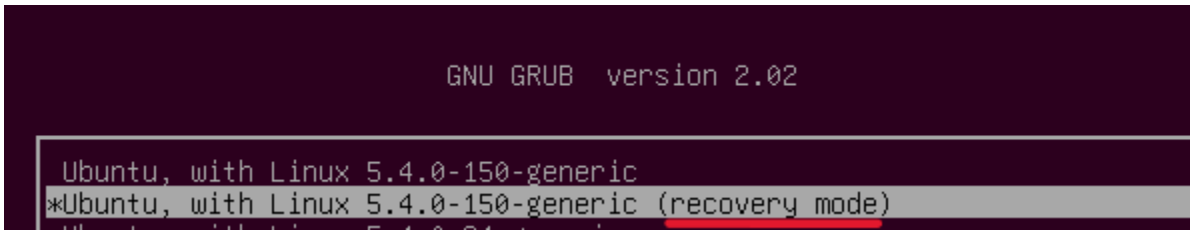
Answer:

Step 1: Enter Recovery Mode

- Restart Ubuntu virtual machine.
- When the Ubuntu logo appears, press:
  - **Left Shift** (VirtualBox)
- The **GRUB menu** will appear.
- Select: Advanced options for Ubuntu



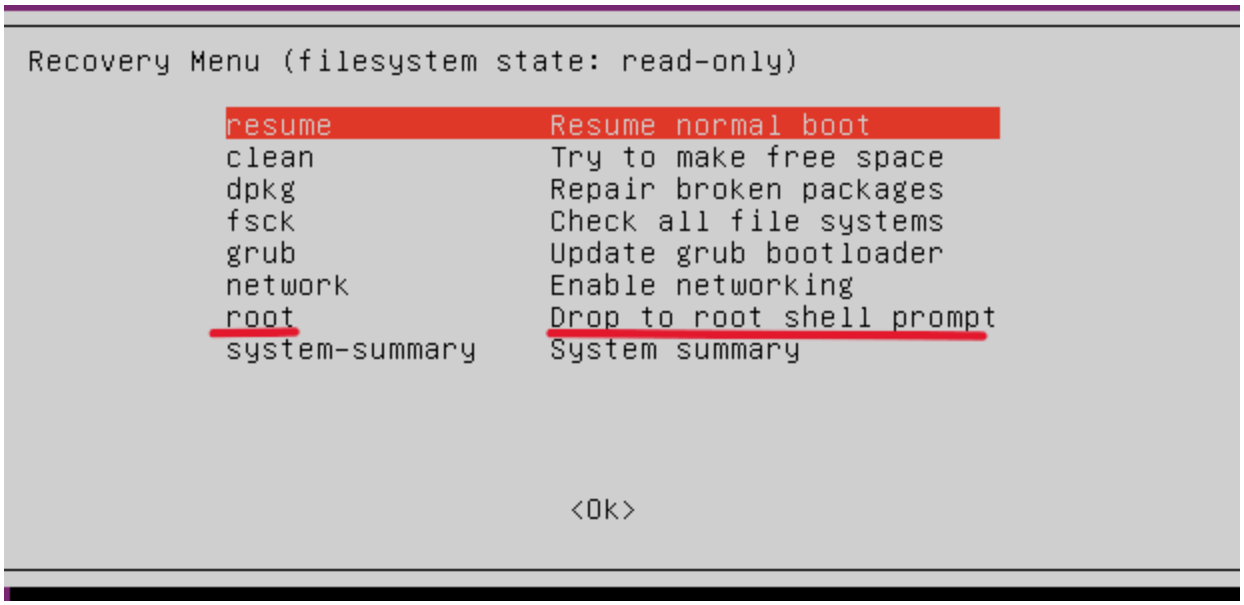
- Select the line that ends with: (recovery mode)



Step 2: Drop to root shell

In the Recovery Menu, choose:

root - Drop to root shell prompt



Step 3: Remount the filesystem as read-write

- Type: **mount -o rw,remount /**

```
Try 'mount --help' for more information.
root@NguyenDuongPhuc:~# mount -o rw,remount /
```

- Explanation:

  In recovery mode, the filesystem is read-only.

  We need **read-write** access to modify user permissions.

Step 4: Add your user to the sudo group

- My username is **yangpuck**, run:

  usermod -aG sudo yangpuck

```
root@NguyenDuongPhuc:~# usermod -aG sudo yangpuck
root@NguyenDuongPhuc:~# _
```

- Explanation:
  - **usermod**: modify a user
  - **-aG**: append to a group
  - **sudo**: group with admin rights

Step 5: Reboot

reboot

Step 6: Install g++

Open a normal terminal and run:

sudo apt update

```
yangpuck@NguyenDuongPhuc:~$ sudo apt update
[sudo] password for yangpuck:
Hit:1 http://vn.archive.ubuntu.com/ubuntu bionic InRelease
Hit:2 http://vn.archive.ubuntu.com/ubuntu bionic-updates InRelease
Hit:3 http://vn.archive.ubuntu.com/ubuntu bionic-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu bionic-security InRelease
Get:5 http://vn.archive.ubuntu.com/ubuntu bionic/main Translation-en_CA [430 kB
]
Get:6 http://vn.archive.ubuntu.com/ubuntu bionic/universe Translation-en_CA [4,
118 kB]
Fetched 4,548 kB in 33s (139 kB/s)
Reading package lists... Done
Building dependency tree
Reading state information... Done
57 packages can be upgraded. Run 'apt list --upgradable' to see them.
yangpuck@NguyenDuongPhuc:~$
```

sudo apt install g++

```
yangpuck@NguyenDuongPhuc:~$ sudo apt install g++
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  g++-7 gcc gcc-7 libasan4 libatomic1 libc-dev-bin libc6 libc6-dbg libc6-dev
  libcilkrts5 libgcc-7-dev libitm1 liblsan0 libmpx2 libquadmath0
  libstdc++-7-dev libtsan0 libubsan0 linux-libc-dev manpages-dev
Suggested packages:
  g++-multilib g++-7-multilib gcc-7-doc libstdc++6-7-dbg gcc-multilib make
  autoconf automake libtool flex bison gcc-doc gcc-7-multilib gcc-7-locales
  libgcc1-dbg libgomp1-dbg libitm1-dbg libatomic1-dbg libasan4-dbg
  liblsan0-dbg libtsan0-dbg libubsan0-dbg libcilkrts5-dbg libmpx2-dbg
  libquadmath0-dbg glibc-doc libstdc++-7-doc
The following NEW packages will be installed:
  g++ g++-7 gcc gcc-7 libasan4 libatomic1 libc-dev-bin libc6-dev libcilkrts5
  libgcc-7-dev libitm1 liblsan0 libmpx2 libquadmath0 libstdc++-7-dev libtsan0
  libubsan0 linux-libc-dev manpages-dev
The following packages will be upgraded:
  libc6 libc6-dbg
2 upgraded, 19 newly installed, 0 to remove and 55 not upgraded.
Need to get 37.9 MB of archives.
After this operation, 118 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://vn.archive.ubuntu.com/ubuntu bionic-updates/main amd64 libc6-dbg a
md64 2.27-3ubuntu1.6 [5,162 kB]
Get:2 http://vn.archive.ubuntu.com/ubuntu bionic-updates/main amd64 libc6 amd64
 2.27-3ubuntu1.6 [2,831 kB]
Get:3 http://vn.archive.ubuntu.com/ubuntu bionic-updates/main amd64 libitm1 amd
```

```
Unpacking manpages-dev (4.15-1) ...
Setting up libquadmath0:amd64 (8.4.0-1ubuntu1~18.04) ...
Setting up libatomic1:amd64 (8.4.0-1ubuntu1~18.04) ...
Setting up libc6-dbg:amd64 (2.27-3ubuntu1.6) ...
Setting up libasan4:amd64 (7.5.0-3ubuntu1~18.04) ...
Setting up libcilkrts5:amd64 (7.5.0-3ubuntu1~18.04) ...
Setting up libubsan0:amd64 (7.5.0-3ubuntu1~18.04) ...
Setting up libtsan0:amd64 (8.4.0-1ubuntu1~18.04) ...
Setting up linux-libc-dev:amd64 (4.15.0-213.224) ...
Setting up liblsan0:amd64 (8.4.0-1ubuntu1~18.04) ...
Setting up libmpx2:amd64 (8.4.0-1ubuntu1~18.04) ...
Setting up libc-dev-bin (2.27-3ubuntu1.6) ...
Setting up manpages-dev (4.15-1) ...
Setting up libc6-dev:amd64 (2.27-3ubuntu1.6) ...
Setting up libitm1:amd64 (8.4.0-1ubuntu1~18.04) ...
Setting up libgcc-7-dev:amd64 (7.5.0-3ubuntu1~18.04) ...
Setting up libstdc++-7-dev:amd64 (7.5.0-3ubuntu1~18.04) ...
Setting up gcc-7 (7.5.0-3ubuntu1~18.04) ...
Setting up g++-7 (7.5.0-3ubuntu1~18.04) ...
Setting up gcc (4:7.4.0-1ubuntu2.3) ...
Setting up g++ (4:7.4.0-1ubuntu2.3) ...
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto m
ode
Processing triggers for man-db (2.8.3-2ubuntu0.1) ...
Processing triggers for libc-bin (2.27-3ubuntu1.5) ...
yangpuck@NguyenDuongPhuc:~$
```

Step 7: Create the C++ file using nano

- Open nano editor:

  nano time.cpp

```
yangpuck@NguyenDuongPhuc:~$ nano time.cpp
```

```
  GNU nano 2.9.3                    time.cpp                          Modified

#include <iostream>
#include <sys/time.h>
#include <sys/wait.h>
#include <unistd.h>

using namespace std;

int main(int argc, char *argv[]) {
    if (argc < 2) {
        cerr << "Usage: " << argv[0] << " <command>\n";
        return 1;
    }

    struct timeval start, end;
    gettimeofday(&start, nullptr);

    pid_t pid = fork();
    if (pid == -1) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        if (execvp(argv[1], &argv[1]) == -1) {
            perror("execvp failed");
            return 1;
        }
    } else {
        // Parent process
        wait(nullptr);
        gettimeofday(&end, nullptr);

        double elapsed_time =
            (end.tv_sec - start.tv_sec) +
            (end.tv_usec - start.tv_usec) / 1000000.0;

        cout << "Execution time: " << elapsed_time << " seconds" << endl;
    }

    return 0;
}

^G Get Help    ^O Write Out   ^W Where Is    ^K Cut Text    ^J Justify
^X Exit        ^R Read File   ^\ Replace     ^U Uncut Text  ^T To Spell
```

- Explanation:
  - The program measures the execution time of a shell command provided by the user.
  - It first checks whether the user entered a command; if not, it prints the correct usage message.
  - The program records the start time using gettimeofday().
  - It creates a new process using fork().
  - **Child process:**
    - Calls execvp() to execute the shell command.
    - If execvp() fails, it prints an error and exits.
  - **Parent process:**
    - Waits for the child process to finish using wait().
    - Records the end time after the command has completed.
  - The program calculates the execution time by subtracting the start and end timestamps (seconds + microseconds).
  - It then prints the total execution time in seconds.
  - Summary: **fork() creates a child → child runs the command with execvp() → parent waits → parent measures and prints the execution time**.

Save and exit nano

Press Ctrl + O → save

Press Ctrl + X → exit

Step 8: Compile the program

g++ time.cpp -o time

```
yangpuck@NguyenDuongPhuc:~$ g++ time.cpp -o time
```

Explanation:

- **g++** = C++ compiler
- **time.cpp** = the source file
- **-o time** = output executable named time

Step 9: Run the program

```
yangpuck@NguyenDuongPhuc:~$ ./time ls
 check_age.sh   Downloads              gradebook.csv   Public      Videos
 Desktop        examples.desktop       Music           Templates
 Documents      find_grade.sh          Myweb           time
'~Documents'    find_string_files.sh   Pictures        time.cpp
Execution time: 0.001733 seconds
yangpuck@NguyenDuongPhuc:~$ ./time sleep 2
Execution time: 2.00603 seconds
```

- Example 1:

  ./time ls

Output:

- First: the result of ls (file list)
- Last line:

Execution time: 0.001733 seconds

- Example 2 (longer command):

  ./time sleep 2

Output:

Execution time: 2.00603 seconds

2. Design a file-copying program named filecopy.cpp using ordinary pipes. This program will be passed two parameters: the name of the file to be copied and the name of the destination file. The program will then create an ordinary pipe and write the contents of the file to be copied to the pipe. The child process will read this file from the pipe and write it to the destination file. For example, if we invoke the program as follows:

./filecopy input.txt copy.txt

The file input.txt will be written to the pipe. The child process will read the contents of this file and write it to the destination file copy.txt.

Answer:

1. Create the source file with nano

- **Command**

  nano filecopy.cpp

```
yangpuck@NguyenDuongPhuc:~$ nano filecopy.cpp
```

- **Code**

```
GNU nano 2.9.3                    filecopy.cpp                    Modified

#include <iostream>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

using namespace std;

int main(int argc, char *argv[]) {
    if (argc != 3) {
        cerr << "Usage: " << argv[0] << " <source> <destination>\n";
        return 1;
    }

    const char* src = argv[1];
    const char* dst = argv[2];

    int fd[2];
    if (pipe(fd) == -1) {
        perror("pipe failed");
        return 1;
    }

    pid_t pid = fork();
```

```
    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid > 0) {
        // Parent process: write to pipe
        close(fd[0]);  // Close unused read end

        int srcFile = open(src, O_RDONLY);
        if (srcFile < 0) {
            perror("cannot open source file");
            return 1;
        }

        char buffer[1024];
        ssize_t bytesRead;
```

```
        while ((bytesRead = read(srcFile, buffer, sizeof(buffer))) > 0) {
            write(fd[1], buffer, bytesRead);
        }

        close(srcFile);
        close(fd[1]);  // Close write end so child knows EOF

        wait(nullptr); // Wait for child
    }
    else {
        // Child process: read and write
        close(fd[1]);  // Close unused write end

        int dstFile = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (dstFile < 0) {
            perror("cannot open destination file");
            return 1;
        }

        char buffer[1024];
        ssize_t bytesRead;
```

```
        while ((bytesRead = read(fd[0], buffer, sizeof(buffer))) > 0) {
            write(dstFile, buffer, bytesRead);
        }

        close(dstFile);
        close(fd[0]);
    }

    return 0;
}

^G Get Help    ^O Write Out   ^W Where Is    ^K Cut Text    ^J Justify
^X Exit        ^R Read File   ^\ Replace     ^U Uncut Text  ^T To Spell
```

- **Explanation**

I. Check arguments

- if (argc != 3)
  → Checks that exactly two parameters are provided: the source file and the destination file.

II. Store filenames

- const char* src = argv[1];
- const char* dst = argv[2];
  → Stores the names of the source and destination files.

III. Create a pipe

- int fd[2];
- pipe(fd);
  → Creates a pipe with two ends:
  - fd[0] = read end
  - fd[1] = write end

IV. Fork

- pid_t pid = fork();
  → Creates a child process.

V. Parent process block (pid > 0)

Close unused pipe end

- close(fd[0]);
  → Parent does not read from the pipe.

Open source file

- open(src, O_RDONLY);
  → Opens the input file for reading.

Read from source file

- read(srcFile, buffer, sizeof(buffer));
  → Reads chunks of data into the buffer.

Write to pipe

- write(fd[1], buffer, bytesRead);
  → Sends the data through the pipe to the child process.

Close descriptors

- close(srcFile);

- close(fd[1]);
  → Closes the file and the write end of the pipe (signals EOF to the child).

Wait for child

- wait(nullptr);
  → Waits for the child process to finish.

VI. Child process block (pid == 0)

Close unused pipe end

- close(fd[1]);
  → Child does not write to the pipe.

Open destination file

- open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
  → Opens (or creates) the destination file.

Read from pipe

- read(fd[0], buffer, sizeof(buffer));
  → Reads the data sent by the parent.

Write to destination file

- write(dstFile, buffer, bytesRead);
  → Writes the data into the output file.

Close descriptors

- close(dstFile);
- close(fd[0]);

VII. End of program

- return 0;

Save & exit

- Press Ctrl + O → Enter (save)
- Press Ctrl + X (exit)

1. Compile with g++

- **Command**

  g++ filecopy.cpp -o filecopy

```
yangpuck@NguyenDuongPhuc:~$ g++ filecopy.cpp -o filecopy
```

- **Explanation**
  - **g++** is the GNU C++ compiler.
  - **filecopy.cpp** is the input source file.
  - **-o filecopy** tells the compiler to write the executable named filecopy.
  - On success you get an executable ./filecopy.

2. Create input.txt quickly using cat and redirection

- **Command**

  cat > input.txt

  hello world

  24521386

[Ctrl + D] (exit)

```
yangpuck@NguyenDuongPhuc:~$ cat > input.txt
hello world
24521386
```

- **Explanation**
  - **cat > input.txt** makes the shell redirect standard input into input.txt.
  - Type lines (e.g., **hello world and 24521386**), then press Ctrl + D to send EOF and finish.
  - **input.txt** is created containing the lines you typed.

3. Run the program to copy the file using an ordinary pipe

- **Command**

  ./filecopy input.txt copy.txt

```
yangpuck@NguyenDuongPhuc:~$ ./filecopy input.txt copy.txt
```

- **Explanation**
  - Runs your program filecopy with two arguments:
    - input.txt = source file to copy
    - copy.txt = destination file to create
  - Parent process reads input.txt and writes bytes into a pipe.
  - Child process reads from the pipe and writes to copy.txt.

4. Verify the copied file with cat

- **Command**

  cat copy.txt

```
yangpuck@NguyenDuongPhuc:~$ cat copy.txt
hello world
24521386
```

- **Explanation**
  - Prints the contents of copy.txt to the terminal so you can visually verify the copy.
  - If it matches input.txt, the program worked.